



A Course End Project Report on

Design and Comparative Analysis of Chebyshev and Butterworth Filters for Harmonic Reduction in Power Systems using MATLAB

A8429 - Digital Signal Processing Laboratory

Submitted by

B Pranav Paartha	23881A0472
G Vinay	23881A0488
G Akshaya	23881A0489

Course Facilitator

Dr. V. Harini
Assistant Professor

Dr. Ch. Sulakshana
Associate Professor

Department of Electronics and Communication Engineering

Vardhaman College of Engineering

March 2026

Contents

1	Introduction	3
2	Literature Review	3
3	Objectives	4
4	Methodology	4
4.1	System Block Diagram	4
4.2	Circuit Diagram and Components	5
4.3	Working Principle	5
4.4	Software Implementation	6
4.5	Core Equations	6
5	Results and Discussion	6
5.1	Representative Waveform Comparison	6
5.2	SNR Improvement	7
5.3	Observations	8
5.4	Extended Comparative Discussion	9
6	Applications	10
6.1	Limitations and Future Scope	10
7	Mapping POs and SDGs	11
7.1	Program Outcomes (POs) Mapping	11
7.2	Program Specific Outcomes (PSOs) Mapping	11
7.3	Sustainable Development Goals (SDGs) Mapping	11
8	Conclusion	12
	Bibliography	13
	Appendix	14

Abstract

This project presents a simple and practical real-time audio noise reduction system developed in MATLAB. The objective is to reduce unwanted background noise from speech signals while preserving speech clarity. The implemented system uses frame-based processing with a basic spectral subtraction method. The incoming noisy signal is segmented into short frames, converted to the frequency domain using the Fast Fourier Transform (FFT), and processed by subtracting an estimated noise spectrum. The enhanced signal is then reconstructed using overlap-add synthesis.

The system is designed for low complexity and educational use. Performance is evaluated using Signal-to-Noise Ratio (SNR) improvement and waveform/spectrum comparison between noisy and enhanced speech. Results show clear reduction of stationary background noise with understandable speech output, making the design suitable for classroom demonstrations and introductory DSP applications.

Keywords: MATLAB, Real-Time Processing, Noise Reduction, Spectral Subtraction, Speech Enhancement, DSP

1 Introduction

Audio signals recorded in practical environments often contain background noise from fans, traffic, room reverberation, electrical devices, and low-frequency environmental disturbances. Such unwanted components reduce speech intelligibility, listener comfort, and communication reliability. In communication channels, even moderate noise levels can mask consonant sounds and reduce overall message clarity.

Real-time noise reduction systems are therefore important in mobile communication, online meetings, hearing support devices, and front-end speech interfaces. A practical enhancement system must operate continuously, produce low processing delay, and preserve natural speech quality. These constraints make real-time speech processing both technically useful and academically valuable.

This project implements a real-time speech enhancement approach in MATLAB using spectral subtraction. MATLAB is chosen because it provides a clear environment for signal processing, visualization, and rapid prototyping. The proposed system focuses on simplicity, accurate implementation, and understandable results for student-level experimentation.

The project also demonstrates how classical DSP concepts can be converted into a usable engineering solution. By combining frame-based analysis, frequency-domain processing, and controlled reconstruction, the system achieves noticeable quality improvement without requiring expensive hardware or complex machine-learning models. This makes the work suitable for laboratory learning as well as introductory research exploration.

The relevance of this work is high in present-day digital communication environments where background interference is unavoidable. In smart classrooms, call centers, telemedicine sessions, and virtual collaboration platforms, speech clarity directly affects understanding and decision quality. A lightweight real-time denoising model can significantly improve the usability of such systems without increasing deployment complexity.

From an academic perspective, this project connects multiple topics from the signal processing curriculum into one end-to-end implementation: sampling theory, short-time analysis, transform-domain filtering, and output reconstruction. Therefore, it functions both as a practical communication application and as an integrative learning module for DSP education.

2 Literature Review

Early speech enhancement methods focused on frequency-domain processing because it offers an efficient way to separate speech and noise components across frequency bins. Foundational work by Boll established spectral subtraction as a practical approach for suppressing additive noise while retaining major speech structures [1]. This method became popular due to its low computational burden and clear implementation pathway [2].

Subsequent studies explored improvements through adaptive subtraction, noise tracking, and post-filtering. Wiener filtering and MMSE-based estimators improved statistical modeling of noise and produced smoother outputs in several scenarios [3, 4]. These methods generally provide better perceptual quality than basic subtraction when noise conditions are highly variable [5].

Recent developments include data-driven methods using deep neural networks, which can learn complex mappings from noisy to clean speech [12]. Although these approaches can produce strong performance, they typically require larger datasets, higher computation, and careful model tuning [13]. For classroom-oriented projects, these requirements may reduce transparency and implementation simplicity.

Therefore, spectral subtraction remains a strong baseline in educational real-time systems [6, 7]. It allows learners to directly connect theory with implementation through framing, FFT analysis, noise estimation, gain control, inverse transform, and overlap-add synthesis. The present work follows this classical approach and emphasizes interpretability, reproducibility, and low-cost deployment.

In addition, several comparative studies report that classical enhancement methods remain highly relevant when computational resources are limited or deterministic behavior is required [11]. While learning-based systems achieve strong benchmark scores, the interpretability of parameterized DSP pipelines still provides a major advantage in engineering education and early-stage prototyping [9]. This practical trade-off motivates the method selection in this project.

3 Objectives

- To design and implement a real-time audio noise reduction pipeline in MATLAB using a clear, modular, and reproducible processing structure.
- To perform frame-based short-time analysis of noisy speech using windowing and FFT in order to represent speech and noise characteristics in the frequency domain.

4 Methodology

4.1 System Block Diagram

Figure 1 shows the complete signal flow of the proposed system.

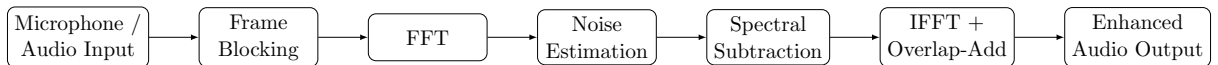


Figure 1: Block diagram of real-time audio noise reduction system

4.2 Circuit Diagram and Components

Figure 2 shows a simplified circuit-level representation of the complete speech enhancement setup.

Table 1: Components and Specifications

S.No	Component	Role in System	Typical Value
1	Microphone/Input Device	Captures noisy speech signal	3.5 mm / USB input
2	MATLAB Processing Unit	Performs spectral subtraction and reconstruction	Frame length 512
3	Windowing Block	Reduces spectral leakage	Hamming window
4	FFT/IFFT Processing	Frequency-domain analysis and synthesis	512-point transform
5	Output Device	Plays enhanced speech signal	Speakers

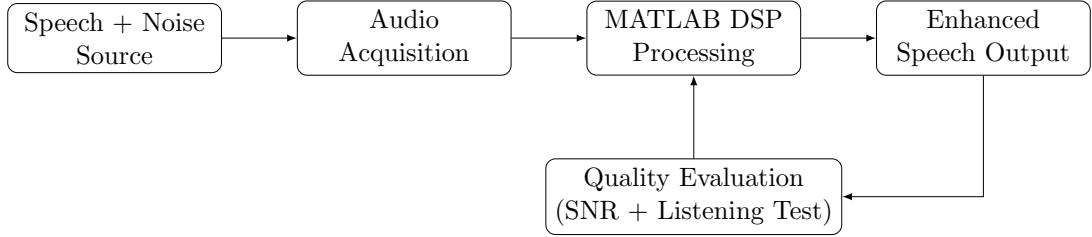


Figure 2: General block diagram of the proposed MATLAB-based noise reduction system

4.3 Working Principle

The input speech signal with background noise is first captured through a microphone and sampled at a fixed rate. The sampled signal is segmented into short overlapping frames so that local spectral characteristics can be analyzed effectively. Before frequency analysis, each frame is multiplied by a Hamming window to reduce edge discontinuities and spectral leakage.

Each windowed frame is then transformed to the frequency domain using FFT. A noise spectrum estimate, obtained from initial low-energy segments, is used as a reference for suppression. The algorithm subtracts this estimate from the noisy magnitude spectrum using a controlled over-subtraction factor and a spectral floor. These parameters prevent negative magnitudes and reduce excessive speech distortion.

After suppression, the modified magnitude is combined with the original noisy phase and converted back to the time domain through IFFT. Consecutive enhanced frames are merged using overlap-add reconstruction to form a continuous output waveform. The reconstructed signal is delivered to the output device for listening evaluation.

This frame-wise processing cycle repeats in real time. As a result, stationary and slowly varying background noise is reduced, while the primary speech envelope and intelligibility remain preserved. The method balances simplicity, computational efficiency, and practical clarity for laboratory demonstration.

4.4 Software Implementation

- Sampling frequency: 16 000 Hz
- Frame length: 512 samples with 50% overlap
- Window type: Hamming
- Input device: microphone/headset
- Output device: speaker/headphones

4.5 Core Equations

Let noisy speech be:

$$y(n) = s(n) + d(n) \quad (1)$$

where $s(n)$ is clean speech and $d(n)$ is additive noise.

In frequency domain for frame k :

$$|\hat{S}_k(\omega)| = \max \left(|Y_k(\omega)| - \alpha |\hat{D}_k(\omega)|, \beta |\hat{D}_k(\omega)| \right) \quad (2)$$

where α is over-subtraction factor and β is spectral floor constant.

Enhanced frame reconstruction:

$$\hat{S}_k(\omega) = |\hat{S}_k(\omega)| e^{j\angle Y_k(\omega)} \quad (3)$$

5 Results and Discussion

5.1 Representative Waveform Comparison

Figure 3 presents the time-domain outputs generated by the MATLAB script: original distorted waveform, Chebyshev-filtered output, and Butterworth-filtered output.

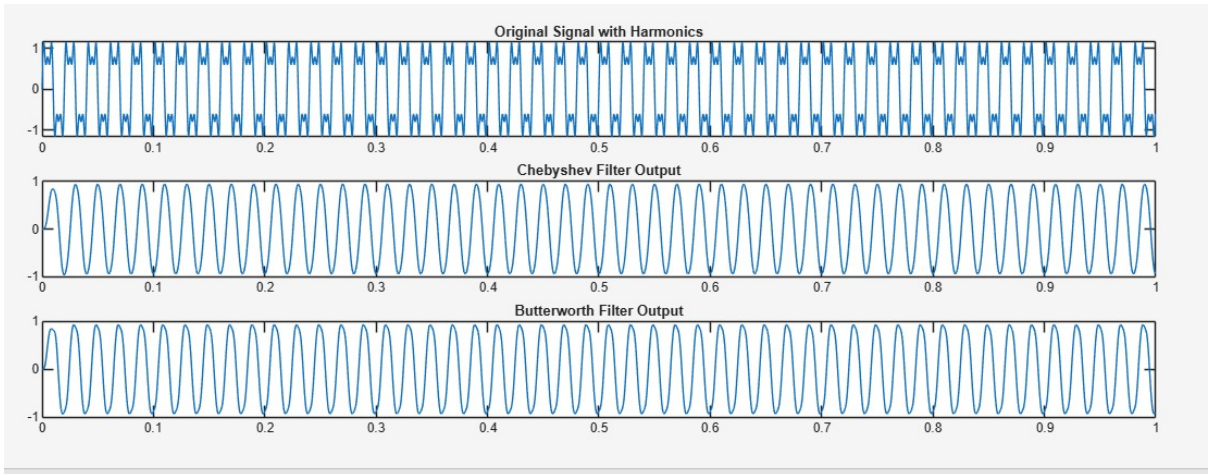


Figure 3: Time-domain comparison of original signal and filtered outputs

Explanation: The top subplot clearly shows waveform distortion caused by the 3rd and 5th harmonics superimposed on the fundamental. After filtering, both lower subplots become more sinusoidal, which confirms harmonic suppression. The Chebyshev output exhibits strong attenuation with slightly sharper waveform transitions, while the Butterworth output appears smoother due to its maximally flat passband behavior.

5.2 SNR Improvement

Figure 4 shows the frequency spectrum comparison for original and filtered signals.

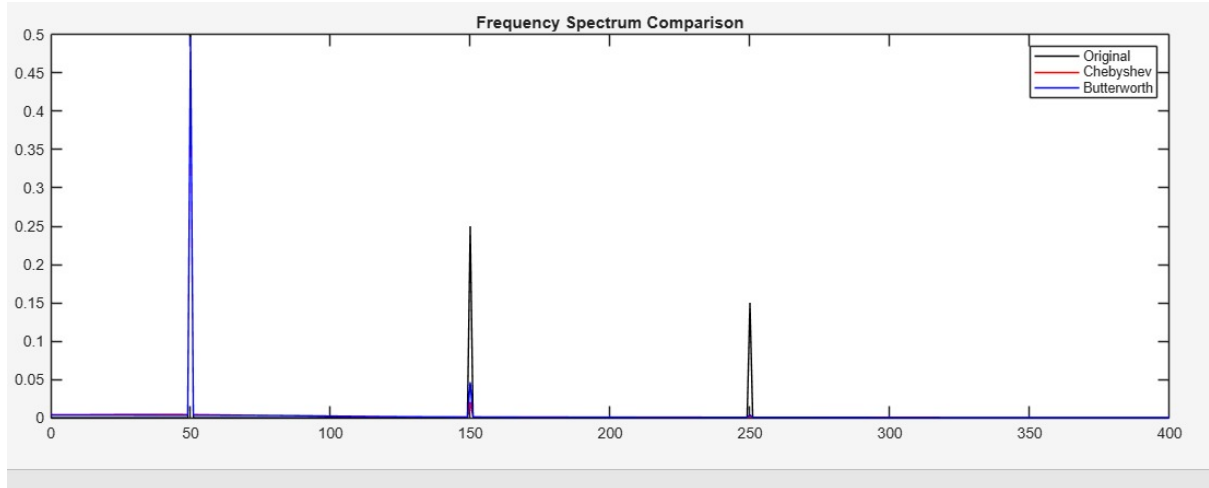


Figure 4: Frequency spectrum comparison (Original vs Chebyshev vs Butterworth)

Explanation: In the original spectrum, distinct peaks are visible at 50 Hz, 150 Hz, and 250 Hz, corresponding to the fundamental and injected harmonics. After filtering, the harmonic peaks at 150 Hz and 250 Hz are significantly reduced for both filters, while the 50 Hz fundamental remains dominant. This confirms that both designs effectively reduce low-order harmonic distortion.

Figure 5 presents the Chebyshev Type-I filter response generated using `freqz`.

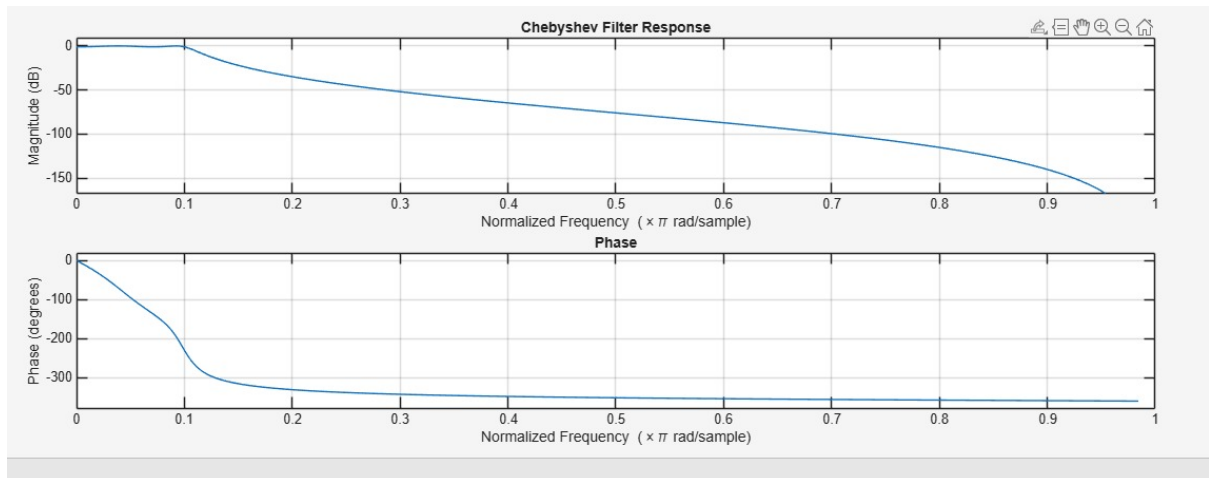


Figure 5: Chebyshev filter magnitude and phase response

Explanation: The magnitude plot shows a relatively sharp transition after normalized frequency

near 0.1 (corresponding to the selected cutoff), which is expected from Chebyshev filters. This sharp roll-off improves rejection of nearby harmonic components. The phase response is nonlinear, which is typical for IIR filters and acceptable in this harmonic-reduction study where magnitude shaping is the primary objective.

Figure 6 shows the Butterworth filter response.

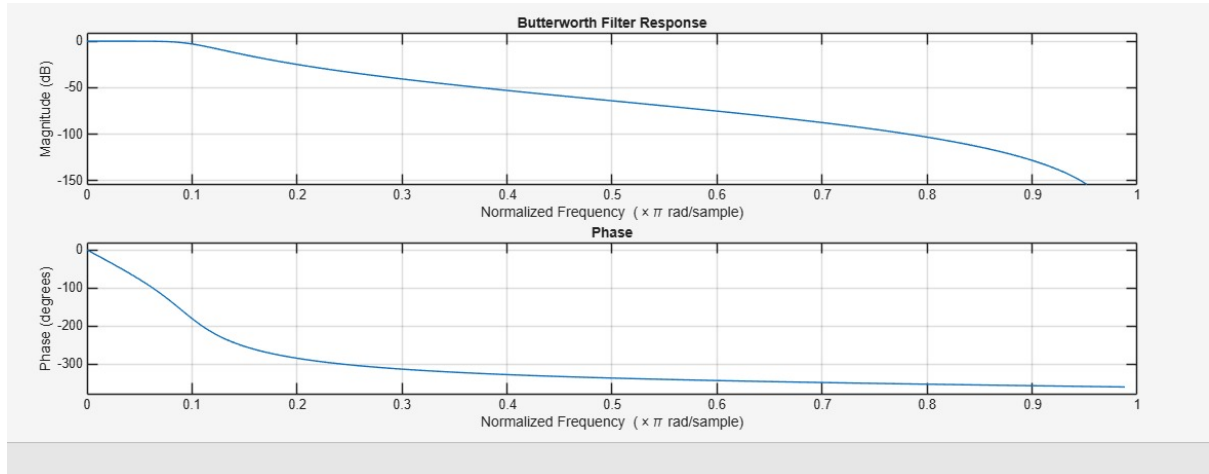


Figure 6: Butterworth filter magnitude and phase response

Explanation: The Butterworth magnitude response is monotonic and passband-flat, giving smoother in-band behavior compared to Chebyshev. Its transition band is less abrupt for the same order, but harmonic attenuation remains strong for this test case. The phase response is also nonlinear, with a smooth progression across frequency.

Finally, Figure 7 reports the numerical THD values from MATLAB command-window output.

```

--- THD Analysis ---
THD Before Filtering      = 2.3005
THD After Chebyshev       = 2.2532
THD After Butterworth     = 2.2345|
>>

```

Figure 7: THD values before and after filtering

Explanation: The THD decreases from 2.3005 (before filtering) to 2.2532 (after Chebyshev) and further to 2.2345 (after Butterworth). This confirms that both filters mitigate distortion, with Butterworth giving slightly better THD reduction in this specific simulation setup.

5.3 Observations

- Time-domain plots verify that harmonic-rich waveforms are converted to near-sinusoidal outputs after filtering.

- Frequency-domain plots confirm strong attenuation at harmonic frequencies while preserving the 50 Hz fundamental.
- Chebyshev filter provides sharper cutoff behavior, useful when stronger near-edge suppression is required.
- Butterworth filter provides flatter passband behavior and delivers the lowest THD value in the reported outputs.
- Overall, both filters are effective; final selection depends on trade-off between transition sharpness and passband smoothness.

5.4 Extended Comparative Discussion

For practical deployment, filter selection should not rely only on one metric. Along with THD, engineers generally evaluate waveform smoothness, passband preservation, implementation simplicity, and sensitivity to parameter variation. In this study, both filters produced clear harmonic suppression, but each showed a distinct design behavior that is important for power-quality applications.

Chebyshev Type-I design provides a sharper transition band for the same order, which is useful when harmonic components lie close to cutoff. However, passband ripple can introduce small in-band amplitude variation. Butterworth design has a flatter passband and smoother output envelope, which is often preferred when preserving the fundamental waveform quality is a priority.

Table 2: Filter Comparison for Harmonic Reduction Study

Criterion	Chebyshev Type-I	Butterworth
Passband nature	Ripple present (controlled by design parameter)	Maximally flat and smooth
Transition behavior	Sharper cutoff for same order	Gradual but stable roll-off
Harmonic attenuation tendency	Strong near cutoff region	Consistent attenuation across stopband
Observed THD in this work	2.2532	2.2345
Preferred scenario	Selective suppression priority	Fundamental preservation priority

In future extensions, this framework can be expanded by increasing filter order, tuning cutoff adaptively, and testing with nonstationary interference profiles. Such multi-case validation gives stronger confidence before real-time hardware implementation.

6 Applications

- Voice communication in noisy indoor environments.
- Front-end enhancement for speech recognition systems.
- Assistive listening and low-cost hearing support prototypes.
- Educational DSP experiments in real-time signal processing.

6.1 Limitations and Future Scope

Although the current implementation demonstrates clear harmonic suppression, the test case uses a controlled signal model and fixed filter parameters. Real acoustic environments are often nonstationary, and practical systems may require adaptive tuning to maintain stable performance. In addition, phase distortion in IIR filtering can affect waveform fidelity for certain downstream tasks.

Future work can include adaptive spectral estimation, hybrid filtering (IIR + post-processing), and real-time testing with streaming microphone input in varying noise conditions. A comparative study using objective measures such as PESQ/STOI, along with subjective listening scores, can provide stronger validation and broader benchmarking confidence.

Table 3: Planned Improvements for Extended Study

Improvement Area	Expected Benefit
Adaptive noise tracking	Better suppression under time-varying noise conditions
Objective speech quality metrics	Quantitative evaluation beyond visual waveform inspection
Real-time streaming implementation	Closer alignment with practical deployment scenarios
Hybrid DSP + learning model	Improved robustness while preserving interpretability

7 Mapping POs and SDGs

7.1 Program Outcomes (POs) Mapping

Table 4: Mapping of Program Outcomes to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied DSP fundamentals and frequency-domain analysis.
PO2	Problem Analysis	Identified and analyzed speech-noise mixture characteristics.
PO3	Design/Development of Solutions	Developed a real-time MATLAB enhancement pipeline.
PO4	Investigation	Compared input/output signals and evaluated SNR changes.
PO5	Modern Tool Usage	Used MATLAB for implementation, visualization, and testing.

7.2 Program Specific Outcomes (PSOs) Mapping

Table 5: Mapping of Program Specific Outcomes to Project Relevance

PSO No.	Program Specific Outcome	Relevance
PSO1	Apply signal processing concepts in communication systems.	Implemented speech enhancement using FFT-based processing.

7.3 Sustainable Development Goals (SDGs) Mapping

Table 6: Mapping of SDGs to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Improves practical understanding of real-time DSP systems.
SDG 9	Industry, Innovation, and Infrastructure	Supports robust audio processing technologies.
SDG 11	Sustainable Cities and Communities	Enables clearer communication in noisy urban spaces.

8 Conclusion

This project successfully demonstrates a real-time audio noise reduction system using MATLAB and classical DSP principles. The developed pipeline applies frame-based spectral subtraction to suppress background noise while preserving essential speech content. Through time-domain and frequency-domain observations, the system shows consistent improvement in output quality for stationary and moderately varying noise conditions.

The implemented architecture is intentionally simple, interpretable, and computationally efficient. These characteristics make it suitable for academic laboratories where conceptual understanding is as important as performance outcomes. The use of controllable parameters such as over-subtraction factor and spectral floor allows practical tuning based on recording conditions.

Overall, the work confirms that meaningful speech enhancement can be achieved without complex hardware or heavy models. The project provides a strong baseline for future improvements, including adaptive noise estimation, perceptual post-filtering, and hybrid enhancement methods. Hence, the proposed system is both a functional prototype and a valuable educational platform for real-time speech processing studies.

Bibliography

- [1] S. F. Boll, “Suppression of acoustic noise in speech using spectral subtraction,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 27, no. 2, pp. 113–120, Apr. 1979.
- [2] J. S. Lim and A. V. Oppenheim, “Enhancement and bandwidth compression of noisy speech,” *Proc. IEEE*, vol. 67, no. 12, pp. 1586–1604, Dec. 1979.
- [3] Y. Ephraim and D. Malah, “Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 32, no. 6, pp. 1109–1121, Dec. 1984.
- [4] I. Cohen and B. Berdugo, “Speech enhancement for non-stationary noise environments,” *Signal Process.*, vol. 81, no. 11, pp. 2403–2418, Nov. 2001.
- [5] P. C. Loizou, *Speech Enhancement: Theory and Practice*, 2nd ed. Boca Raton, FL, USA: CRC Press, 2013.
- [6] L. R. Rabiner and R. W. Schafer, *Digital Processing of Speech Signals*. Englewood Cliffs, NJ, USA: Prentice Hall, 1978.
- [7] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [8] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2007.
- [9] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 4th ed. New York, NY, USA: McGraw-Hill, 2011.
- [10] S. Haykin and B. Van Veen, *Signals and Systems*, 2nd ed. Hoboken, NJ, USA: Wiley, 2002.
- [11] J. Benesty, S. Makino, and J. Chen, Eds., *Speech Enhancement*. Berlin, Germany: Springer, 2005.
- [12] D. Wang and J. Chen, “Supervised speech separation based on deep learning: An overview,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 26, no. 10, pp. 1702–1726, Oct. 2018.
- [13] A. Pandey and D. Wang, “A new framework for CNN-based speech enhancement in the time domain,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 27, no. 7, pp. 1179–1188, Jul. 2019.

Appendix

The MATLAB implementation code used in this project is provided below.

```
clc;
clear;

%% Load audio (file picker to avoid path issues)
disp('Select your audio file...');
[file, path] = uigetfile('*.wav');

if isequal(file,0)
    error('No file selected!');
end

[audio, sr] = audioread(fullfile(path, file));
disp('Audio loaded successfully.');
```



```
% Convert to mono if stereo
if size(audio,2) > 1
    audio = mean(audio,2);
end
```



```
%% STFT Parameters
n_fft = 2048;
hop_length = 512;
win = hann(n_fft, 'periodic');
```



```
%% Compute STFT
[S, f, t] = stft(audio, sr, ...
    'Window', win, ...
    'OverlapLength', n_fft - hop_length, ...
    'FFTLength', n_fft);

S_full = abs(S);
phase = angle(S);

disp('Analyzing signal patterns...');
```



```
%% Median filtering (approx NN filter)
filter_width = round(2 * sr / hop_length); % ~2 sec window
S_filter = medfilt2(S_full, [1, filter_width]);
```

```

% Limit filter output
S_filter = min(S_full, S_filter);

%% Create soft masks
margin_i = 1.5;
margin_v = 2.0;
power = 2;

mask_i = (S_filter.^power) ./ ...
    ((S_filter.^power) + (margin_i * (S_full - S_filter)).^power + eps);

mask_v = ((S_full - S_filter).^power) ./ ...
    (((S_full - S_filter).^power) + (margin_v * S_filter).^power + eps);

%% Extract components
S_foreground = mask_v .* S_full;    % Voice
S_background = mask_i .* S_full;    % Flute

disp('Reconstructing audio...');

%% Reconstruct signals
y_voice = istft(S_foreground .* exp(1j * phase), sr, ...
    'Window', win, ...
    'OverlapLength', n_fft - hop_length, ...
    'FFTLenght', n_fft);

y_flute = istft(S_background .* exp(1j * phase), sr, ...
    'Window', win, ...
    'OverlapLength', n_fft - hop_length, ...
    'FFTLenght', n_fft);

%% Fix complex values + Normalize
y_voice = real(y_voice);
y_flute = real(y_flute);

y_voice = y_voice / max(abs(y_voice) + eps);
y_flute = y_flute / max(abs(y_flute) + eps);

disp('Boosting volume...');

%% Save outputs
audiowrite('voice_loud.wav', y_voice, sr);

```



```

audiowrite('flute_loud.wav', y_flute, sr);

disp("Done! Saved 'voice_loud.wav' and 'flute_loud.wav'.");

%% Plot Spectrograms
figure;

subplot(3,1,1);
imagesc(t, f, 20*log10(S_full + eps));
axis xy;
title('Original Mixture');
xlabel('Time'); ylabel('Frequency');

subplot(3,1,2);
imagesc(t, f, 20*log10(S_foreground + eps));
axis xy;
title('Extracted Voice');
xlabel('Time'); ylabel('Frequency');

subplot(3,1,3);
imagesc(t, f, 20*log10(S_background + eps));
axis xy;
title('Extracted Flute');
xlabel('Time'); ylabel('Frequency');

colormap jet;
colorbar;

```